

Status	Finished
Started	Monday, 17 November 2025, 2:00 PM
Completed	Monday, 17 November 2025, 3:01 PM
Duration	1 hour 1 min
Grade	9.50 out of 30.00 (31.67%)

Question 1

Not answered

Marked out of 3.00

v1 (latest)

Describe two differences between the syntax of Java and the syntax of Lox as described in the book "Crafting Interpreters". Be precise with your terms and explain the differences in some detail for full marks.

There are many you could point out, here are the two I have chosen. Lox used the `fun` keyword to introduce a function definition while Java uses the return type followed by function name and parameters. I realise Java actually relies on the full signature which I guess is a tougher parsing job than Lox has - it just sees `fun` and knows to parse a function definition. Lox also has no real standard library whereas Java has a substantial runtime. Lox supports just a `print` function and the run-time is compiled into every program. In Java the run-time is distributed separately in the JDK and not included in the compiled program.

Response history

Step	Time	Action	State	Marks
1	17/11/25, 14:00:15	Started	Not yet saved	
2	17/11/25, 15:01:42	Attempt finished	Not answered	

Question 2

Not answered

Marked out of 3.00

v3 (of 4)

Write a scanner that will support new types of comments. These comments should start with # and proceed to the end of the line only.

We have populated the answer box with an empty scanner (`QScanner`) which inherits from the `lox` scanner described in the text.

Override whatever methods you need to achieve the desired result.

Response history

Step	Time	Action	State	Marks
1	17/11/25, 14:00:15	Started	Not yet saved	
2	17/11/25, 15:01:42	Attempt finished	Not answered	

Question 3

Correct

Mark 1.00 out of 1.00

v1 (latest)

Which of the following best describes the primary purpose of a scanner in the context of an interpreter?

- ☐ a. To evaluate expressions in the source code.
- ☐ b. To optimize the source code for performance.
- ☒ c. To convert the source code into a sequence of tokens. ✓
- ☐ d. To parse the tokens into an abstract syntax tree (AST).

The correct answer is: To convert the source code into a sequence of tokens.

Response history

Step	Time	Action	State	Marks
<u>1</u>	17/11/25, 14:00:15	Started	Not yet saved	
<u>2</u>	17/11/25, 14:05:45	Saved: To convert the source code into a sequence of tokens.	Answer saved	
3	17/11/25, 15:01:42	Attempt finished	Correct	1.00

Question 4

Correct

Mark 1.00 out of 1.00

v1 (latest)

In the context of designing an AST for the expression $1 + 2 * 3$, which of the following correctly represents the structure, assuming arithmetic operator precedence is respected?

- ☐ a. A flat structure with "+" at the root and "1", "2", and "3" as its children.
- ☒ b. A root node "+" with a left child "1" and a right child "*" (whose children are "2" and "3"). ✓
- ☐ c. Three separate root nodes for "+", "*", and the literals "1", "2", and "3", with no hierarchy.
- ☐ d. A root node "*" with a left child "+" (whose children are "1" and "2") and a right child "3".

The correct answer is: A root node "+" with a left child "1" and a right child "*" (whose children are "2" and "3").

Response history

Step	Time	Action	State	Marks
1	17/11/25, 14:00:15	Started	Not yet saved	
2	17/11/25, 14:05:45	Saved: A root node "+" with a left child "1" and a right child "*" (whose children are "2" and "3").	Answer saved	
3	17/11/25, 15:01:42	Attempt finished	Correct	1.00

Question 5

Correct

Mark 1.00 out of 1.00

v1 (latest)

Upon parsing a unary operator and it's operand such as "-", what form of Abstract Syntax Tree? would fit best into the Lox interpreter described in "Crafting Interpreters"?

- ☐ a. Two separate nodes: one for "-" and one for "a", with no parent-child relationship.
- ☒ b. A parent node labeled "-" with a single child node representing "a". ✓
- ☐ c. A single node containing the string "-a".
- ☐ d. A parent node labeled "-" with two child nodes, both representing "a".

The correct answer is: A parent node labeled "-" with a single child node representing "a".

Response history

Step	Time	Action	State	Marks
<u>1</u>	17/11/25, 14:00:15	Started	Not yet saved	
<u>2</u>	17/11/25, 14:05:45	Saved: A parent node labeled "-" with a single child node representing "a".	Answer saved	
3	17/11/25, 15:01:42	Attempt finished	Correct	1.00

Question 6

Partially correct

Mark 0.50 out of 1.00

v2 (latest)

Match the term to the definition of that term which best matches it from the given set

non-associative	sets order explicitly	✗
associativity	determines which operator is evaluated first when the same operator appears in one expression	✓
parenthesis	operators you can't use in a place where associativity is needed to resolve ambiguity	✗
precedence	determines which operator is evaluated first when there is a mix of operators	✓

The correct answer is: non-associative → operators you can't use in a place where associativity is needed to resolve ambiguity, associativity → determines which operator is evaluated first when the same operator appears in one expression, parenthesis → sets order explicitly, precedence → determines which operator is evaluated first when there is a mix of operators

Response history

Step	Time	Action	State	Marks
<u>1</u>	17/11/25, 14:00:15	Started	Not yet saved	
<u>2</u>	17/11/25, 14:14:43	Saved: non-associative -> sets order explicitly; associativity -> determines which operator is evaluated first when the same operator appears in one expression; parenthesis -> operators you can't use in a place where associativity is needed to resolve ambiguity; precedence -> determines which operator is evaluated first when there is a mix of operators	Answer saved	
3	17/11/25, 15:01:42	Attempt finished	Partially correct	0.50

Question 7

Incorrect

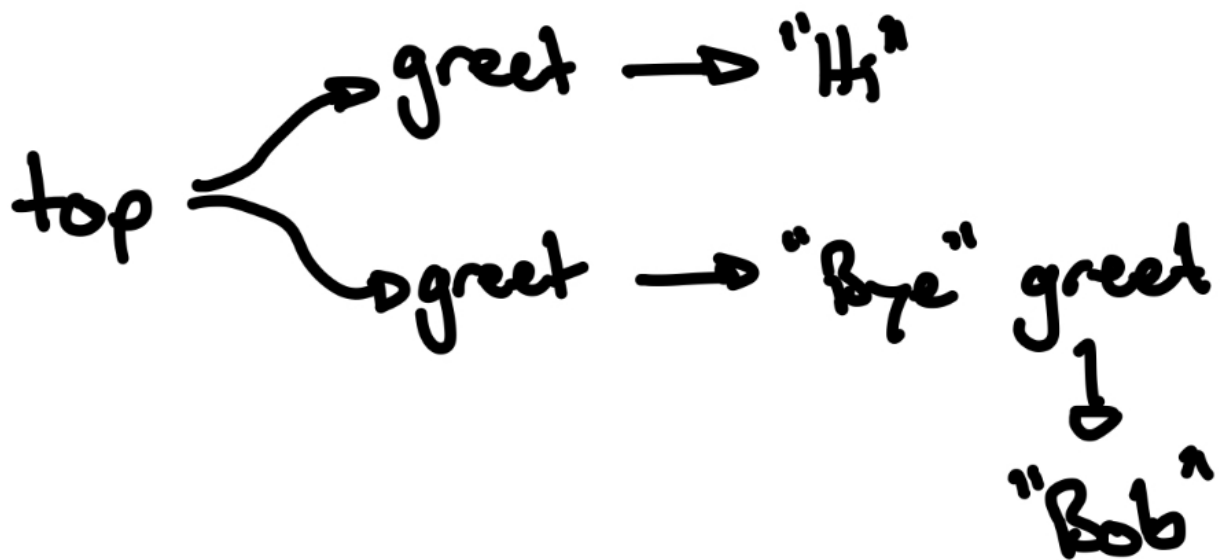
Mark 0.00 out of 1.00

v4 (latest)

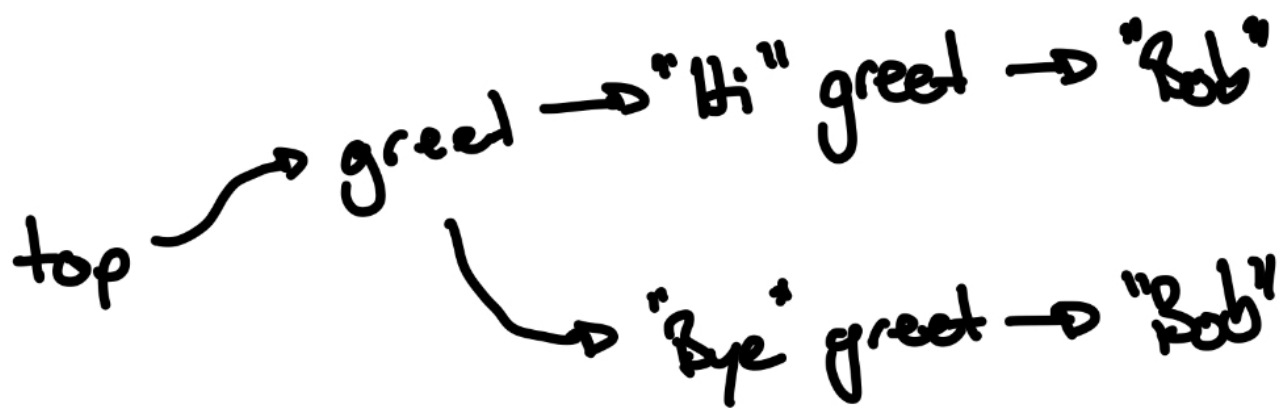
Which of the following is a derivation of the string "Hi Bob Bye Bob " according to this grammar

```
top -> thing*  
greet -> "Hi " greet  
| "Bye " greet  
| person  
person -> "Bob " | "Pete"
```

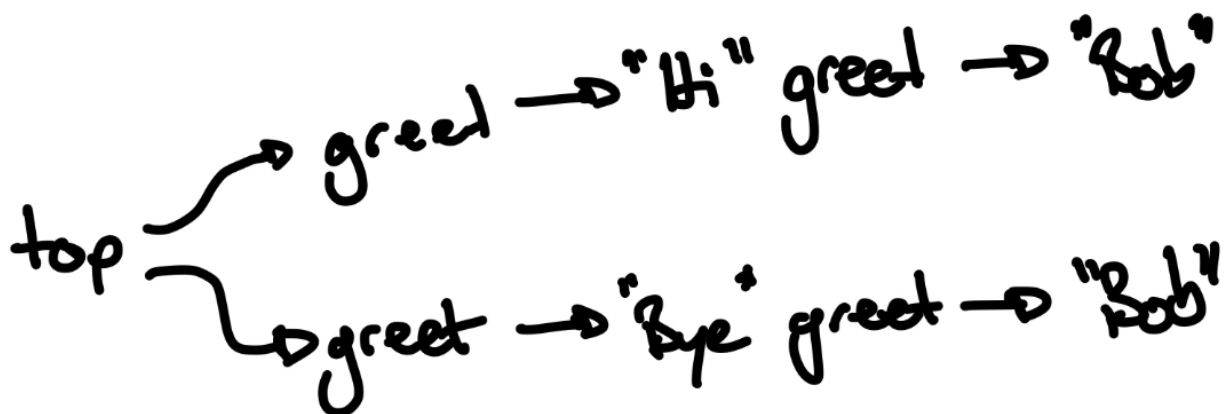
☐ a.



☐ b.



☐ c.



☒ d.



top → greet → "Hi" greet → "Bob"
→ "Bye" greet → "Bob"

The correct answer is:

top → greet → "Hi" greet → "Bob"
→ greet → "Bye" greet → "Bob"

Response history

Step	Time	Action	State	Marks
<u>1</u>	17/11/25, 14:00:15	Started	Not yet saved	
<u>2</u>	17/11/25, 14:14:43	Saved:	Answer saved	
3	17/11/25, 15:01:42	Attempt finished	Incorrect	0.00

Question 8

Correct

Mark 1.00 out of 1.00

v2 (latest)

When evaluating binary operators (e.g., $+$, $*$) in an interpreter such as the Lox interpreter defined in `Interpreter.java`, which of the following steps is typically required?

- ☒ a. Evaluate both operands first, then apply the operator to the results. ✓
- ☐ b. Apply the operator to the first operand only and ignore the second operand.
- ☐ c. Apply the operator directly to the raw syntax of the operands.
- ☐ d. Assume the operands are integers and apply the operator without validation.

The correct answer is: Evaluate both operands first, then apply the operator to the results.

Response history

Step	Time	Action	State	Marks
<u>1</u>	17/11/25, 14:00:15	Started	Not yet saved	
<u>2</u>	17/11/25, 14:14:43	Saved: Evaluate both operands first, then apply the operator to the results.	Answer saved	
3	17/11/25, 15:01:42	Attempt finished	Correct	1.00

Question 9

Complete

Mark 0.00 out of 3.00

v1 (of 2)

Imagine you are writing your own general purpose programming language which includes the following datatypes:

- integers
- booleans
- characters
- strings

It has been decided that *any* value can be tested for "truthyness". Provide rules for each datatype with an explanation for why you think your rule will work best for programmers.

Note: this language will *not* have a void/null/None/nil value, everything is a value-type and there are no reference-types.

Rules for datatypes:

1.Integer: For integer, parser will identify it by number and terminate the lookahead process with the appearance of (.) or a blank space. For blank space it will evaluate the value of the integer and for the appearance of the (.), the parser will consider it as a double or ObjectType according to the character appears after (.)

2.Booleans: The parser will look for TRUE or FALSE or (=) for booleans. The = sign followed by a blank space will not be counted as a boolean type. In this case, it will be a double lookahead for the (=) sign followed by another(=) or (!).

3. Characters: The characters will be tested by parser with the (') sign and looking for the appearance of another (') sign.

4.Strings: It is same as for characters while the parser will evaluate it with (") sign.

In this rules, the parser will stop evaluating as soon as a blank space appears. It will help handle the void cases. Also the evaluating will not start if the trigger (e.g, (") sign for strings) does not appear. It will help handle the null cases.

Comment:

Response history

Step	Time	Action	State	Marks
<u>1</u>	17/11/25, 14:00:15	Started	Not yet saved	

Step	Time	Action	State	Marks
<u>2</u>	17/11/25, 14:52:53	Saved: Rules for datatypes: 1.Integer: For integer, parser will identify it by number and terminate the lookahead process with the appearance of (.) or a blank space. For blank space it will evaluate the value of the integer and for the appearance of the (.), the parser will consider it as a double or ObjectType according to the character appears after (.) 2.Booleans: The parser will look for TRUE or FALSE or (=) for booleans. The = sign followed by a blank space will not be counted as a boolean type. In this case, it will be a double lookahead for the (=) sign followed by another(=) or (!). 3. Characters: The characters will be tested by parser with the (') sign and looking for the appearance of another (') sign. 4.Strings: It is same as for characters while the parser will evaluate it with (") sign. In this rules, the parser will stop evaluating as soon as a blank space appears. It will help handle the void cases.	Answer saved	
<u>3</u>	17/11/25, 15:01:39	Saved: Rules for datatypes: 1.Integer: For integer, parser will identify it by number and terminate the lookahead process with the appearance of (.) or a blank space. For blank space it will evaluate the value of the integer and for the appearance of the (.), the parser will consider it as a double or ObjectType according to the character appears after (.) 2.Booleans: The parser will look for TRUE or FALSE or (=) for booleans. The = sign followed by a blank space will not be counted as a boolean type. In this case, it will be a double lookahead for the (=) sign followed by another(=) or (!). 3. Characters: The characters will be tested by parser with the (') sign and looking for the appearance of another (') sign. 4.Strings: It is same as for characters while the parser will evaluate it with (") sign. In this rules, the parser will stop evaluating as soon as a blank space appears. It will help handle the void cases. Also the evaluating will not start if the trigger (e.g, (") sign for strings) does not appear. It will help handle the null cases.	Answer saved	
<u>4</u>	17/11/25, 15:01:42	Attempt finished	Complete	
5	19/11/25, 13:45:36	Manually graded 0 with comment: <u>Matthew Roberts</u>	Complete	0.00

Question 10

Incorrect

Mark 0.00 out of 1.00

v1 (latest)

Match each of these Lox concepts with the phase of the interpreter they fit into

Expr Object TokenType 

Your answer is incorrect.

The correct answer is: Expr → Parser, Object → Interpreter, TokenType → Scanner

Response history

Step	Time	Action	State	Marks
<u>1</u>	17/11/25, 14:00:15	Started	Not yet saved	
<u>2</u>	17/11/25, 14:14:43	Saved: Expr -> Interpreter; Object -> Parser; TokenType -> Scanner	Answer saved	
<u>3</u>	17/11/25, 14:52:53	Saved: Expr -> Interpreter; Object -> Scanner; TokenType -> Parser	Answer saved	
4	17/11/25, 15:01:42	Attempt finished	Incorrect	0.00

Question 11

Correct

Mark 1.00 out of 1.00

v1 (latest)

In the following code, which lines contain *variable declarations*. Don't include variable *uses* in your answer

```
var x = 1;  
print x;  
print x + 1;
```

- ☐ a. 2 and 3
- ☒ b. 1 only ✓
- ☐ c. 1, 2, and 3
- ☐ d. 3 only
- ☐ e. none
- ☐ f. 2 only

The correct answer is: 1 only

Response history

Step	Time	Action	State	Marks
<u>1</u>	17/11/25, 14:00:15	Started	Not yet saved	
<u>2</u>	17/11/25, 14:23:12	Saved: 1 only	Answer saved	
3	17/11/25, 15:01:42	Attempt finished	Correct	1.00

Question 12

Correct

Mark 1.00 out of 1.00

v1 (latest)

In the following code, which lines contain *variable uses*. Don't include variable *declarations* in your answer

```
var x = 1;  
print x;  
print x + 1;
```

- ☒ a. 2 and 3 ✓
- ☐ b. 2 only
- ☐ c. none
- ☐ d. 1, 2, and 3
- ☐ e. 1 only
- ☐ f. 3 only

The correct answer is: 2 and 3

Response history

Step	Time	Action	State	Marks
<u>1</u>	17/11/25, 14:00:15	Started	Not yet saved	
<u>2</u>	17/11/25, 14:23:12	Saved: 2 and 3	Answer saved	
3	17/11/25, 15:01:42	Attempt finished	Correct	1.00

Question 13

Not answered

Marked out of 3.00

v1 (of 3)

Write some lox code (it can use lox features we haven't implemented yet if you like) which shows some of the "strange" or "interesting" things you can do because assignment is an expression in our parser. What is the output expected of this code?

```
while (x = x - 1){  
  print x;  
}
```

Assignment in lox is an expression whose value is whatever value got assigned. In this case it is $x-1$ every time we go around the loop. Lox also treats all integers as "truthy" so the value never trips the loop to stop. Thus this loop will print from x to negative infinity.

Response history

Step	Time	Action	State	Marks
1	17/11/25, 14:00:15	Started	Not yet saved	
2	17/11/25, 15:01:42	Attempt finished	Not answered	

Question 14

Incorrect

Mark 0.00 out of 1.00

v2 (latest)

The following is the grammar we used to parse logical expressions but the *and* and *or* parts are missing

```

program  -> declaration* EOF;
declaration -> varDecl
| statement;
statement -> exprStmt
| forStmt
| ifStmt
| printStmt
| whileStmt
| block;
block    -> "{" definition* "}"
exprStmt -> expression ";"
forStmt  -> "for" "(" ( varDecl | exprStmt | ";" )
expression? ";"
expression? ")" statement ;
ifStmt   -> "if" "(" expression ")" statement
( "else" statement )? ;
printStmt -> "print" expression ";"
whileStmt -> "while" "(" expression ")" statement ;
varDecl  -> "var" IDENTIFIER ( "=" expression )? ";"
expression -> assignment;
assignment -> IDENTIFIER "=" assignment
| logic_or;
logic_or   -> ??? ;
logic_and  -> ???;
equality   -> comparison ( ("!=" | "==" ) comparison ) * ;
comparison -> term ( ( ">" | ">=" | "<" | "<=" ) term ) * ;
term       -> factor ( ( "-" | "+" ) factor ) * ;
factor     -> unary ( ( "/" | "*" ) unary ) * ;
unary      -> ( "!" | "-" ) unary
| primary ;
primary    -> NUMBER | STRING | "true" | "false" | "nil"
| "(" expression ")"
| IDENTIFIER;

```

Which of the following ways to fill-in the missing parts will give a grammar where *the and operator* is given as an infix **&&** and *the or operator* is given as an infix **||** as in C. *The and operator* must have higher precedence than *the or operator*.

- ☒ a. `logic_or -> equality ( "||" equality)* ; logic_and -> logic_or ( "&&" logic_or)*;` 
- ☐ b. `logic_or -> logic_and ( "or" logic_and)* ; logic_and -> equality ( "and" equality)*;`
- ☐ c. `logic_or -> logic_and ( "||" logic_and)* ; logic_and -> equality ( "&&" equality)*;`
- ☐ d. `logic_or -> equality ( "or" equality)* ; logic_and -> logic_or ( "and" logic_or)*;`

The correct answer is: `logic_or -> logic_and ( "||" logic_and)* ; logic_and -> equality ( "&&" equality)*;`

Response history

Step	Time	Action	State	Marks
<u>1</u>	17/11/25, 14:00:15	Started	Not yet saved	

Step	Time	Action	State	Marks
<u>2</u>	17/11/25, 14:23:12	Saved: logic_or -> equality (" " equality)* ;logic_and -> logic_or ("&&" logic_or)*;	Answer saved	
3	17/11/25, 15:01:42	Attempt finished	Incorrect	0.00

Question 15

Incorrect

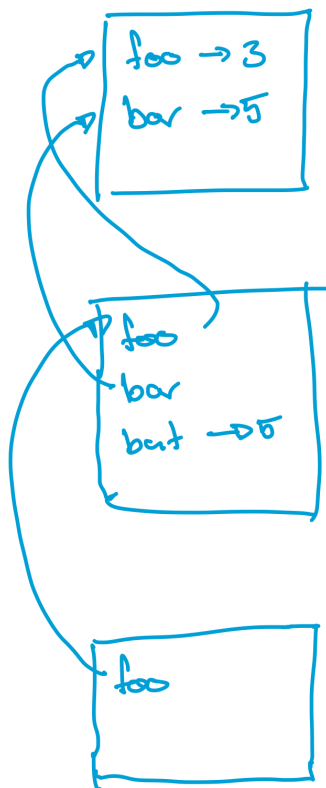
Mark 0.00 out of 2.00

v2 (latest)

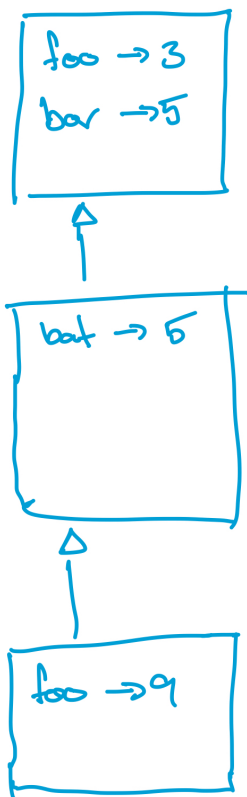
Which environment most accurately represents the state of the environment *just before* line 9 is evaluated in the following Lox program given the environment is defined as explained in "Crafting Interpreters"?

```
1> var foo = 3;
2> var bar;
3> bar = foo;
4> {
5>   var bat = 5;
6>   bar = bat;
7>   {
8>     var foo = 9;
9>     bat = foo;
10>  }
11> }
```

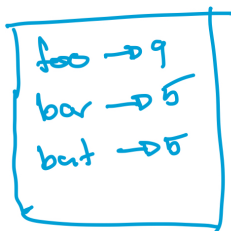
☐ a.



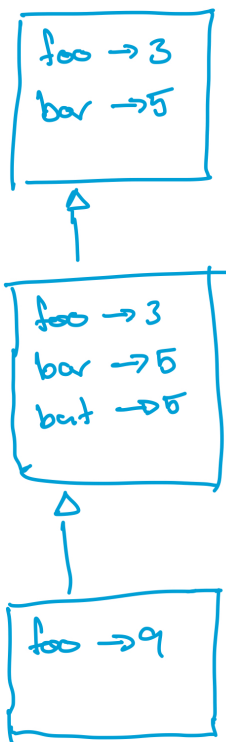
☐ b.



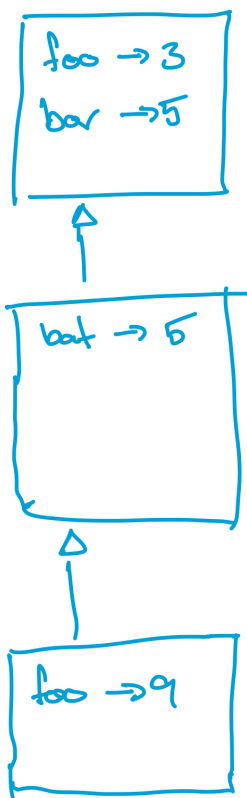
☐ c.



d.



Your answer is incorrect.



The correct answer is:

Response history

Step	Time	Action	State	Marks
------	------	--------	-------	-------

Step	Time	Action	State	Marks
<u>1</u>	17/11/25, 14:00:15	Started	Not yet saved	
<u>2</u>	17/11/25, 14:23:12	Saved: []	Answer saved	
3	17/11/25, 15:01:42	Attempt finished	Incorrect	0.00

Question 16

Correct

Mark 1.00 out of 1.00

v2 (latest)

Given the following lox program, what is the output if the last line is replaced with `print(one(bat)(0));`

```
fun foo(z){
  fun lam(x){
    return x;
  }
  return lam;
}

fun bar(z){
  fun lam(x){
    return z(x);
  }
  return lam;
}

fun bat(x){
  return x+1;
}

fun one(x){return x;}

fun baz(m){
  fun lam1(n){
    fun lam2(f){
      fun lam3(x){
        return m(f)(n(f)(x));
      }
      return lam3;
    }
    return lam2;
  }
  return lam1;
}

print("last line");
```

- ☐ a. 2
- ☐ b. a run time error
- ☒ c. 1 ✓
- ☐ d. ""
- ☐ e. 0

The correct answer is: 1

Response history

Step	Time	Action	State	Marks
<u>1</u>	17/11/25, 14:00:15	Started	Not yet saved	
<u>2</u>	17/11/25, 14:33:28	Saved: 1	Answer saved	

Step	Time	Action	State	Marks
3	17/11/25, 15:01:42	Attempt finished	Correct	1.00

Question 17

Correct

Mark 1.00 out of 1.00

v1 (latest)

What is the correct sequence of steps to parse a function header in Lox?

- ☐ a. Parse the function name, then the return type, followed by the block for the body.
- ☐ b. Parse the block for the body first, then the function name, followed by the parameter list.
- ☐ c. Parse the parameter list, then the function name, followed by a block for the body.
- ☒ d. Parse the function name, then the parameter list, followed by a block for the body. ✓

The correct answer is: Parse the function name, then the parameter list, followed by a block for the body.

Response history

Step	Time	Action	State	Marks
1	17/11/25, 14:00:15	Started	Not yet saved	
2	17/11/25, 14:33:28	Saved: Parse the function name, then the parameter list, followed by a block for the body.	Answer saved	
3	17/11/25, 15:01:42	Attempt finished	Correct	1.00

Question 18

Correct

Mark 1.00 out of 1.00

v1 (latest)

What happens to the environment when a function is called in the interpreter?

- ☐ a. The caller's environment is copied into a new frame and used for execution.
- ☐ b. The current environment is reused, and the parameters are overwritten with the argument values.
- ☐ c. No environment changes occur; variables are resolved globally.
- ☒ d. A new environment is created with the function's parameters bound to the argument values. ✓

The correct answer is: A new environment is created with the function's parameters bound to the argument values.

Response history

Step	Time	Action	State	Marks
<u>1</u>	17/11/25, 14:00:15	Started	Not yet saved	
<u>2</u>	17/11/25, 14:33:28	Saved: A new environment is created with the function's parameters bound to the argument values.	Answer saved	
3	17/11/25, 15:01:42	Attempt finished	Correct	1.00

Question 19

Incorrect

Mark 0.00 out of 1.00

v2 (latest)

What kind of value is being returned by the `return` statement in the following code

```
fun foo(){  
  var i = 0;  
  fun bar(){  
    i = i + 1;  
  }  
  return bar;  
}
```

- ☐ a. number
- ☐ b. uninterpreted function
- ☐ c. string
- ☒ d. nil 

The correct answer is: uninterpreted function

Response history

Step	Time	Action	State	Marks
<u>1</u>	17/11/25, 14:00:15	Started	Not yet saved	
<u>2</u>	17/11/25, 14:33:28	Saved: nil	Answer saved	
3	17/11/25, 15:01:42	Attempt finished	Incorrect	0.00

Question 20

Incorrect

Mark 0.00 out of 2.00

v7 (latest)

Only one of the following programs would have an error if we had not fixed the name resolution to account for lexical scopes. Which one?

- ☐ a.
- ```
var a = 5;
{
 fun foo(a) {
 return a + a;
 }

 print(foo(a));
 var b = 2;
 print(foo(b));
}
```
- ☒ b. ✖
- ```
var a = 5;
{
    fun foo() {
        return a + a;
    }

    print(foo());
    var b = 2;
    print(foo());
}
```
- ☐ c.
- ```
var a = 5;
{
 fun foo(a) {
 return a + a;
 }

 print(foo(a));
 var a = 2;
 print(foo(a));
}
```
- ☐ d.
- ```
var a = 5;
{
    fun foo() {
        return a + a;
    }

    print(foo());
    var a = 2;
    print(foo());
}
```

The correct answer is:

```
var a = 5;
{
    fun foo() {
        return a + a;
    }

    print(foo());
    var a = 2;
    print(foo());
}
```

Response history

Step	Time	Action	State	Marks
<u>1</u>	17/11/25, 14:00:15	Started	Not yet saved	
<u>2</u>	17/11/25, 14:33:28	Saved: <pre>var a = 5; { fun foo() { return a + a; } print(foo()); var b = 2; print(foo()); }</pre>	Answer saved	
3	17/11/25, 15:01:42	Attempt finished	Incorrect	0.00